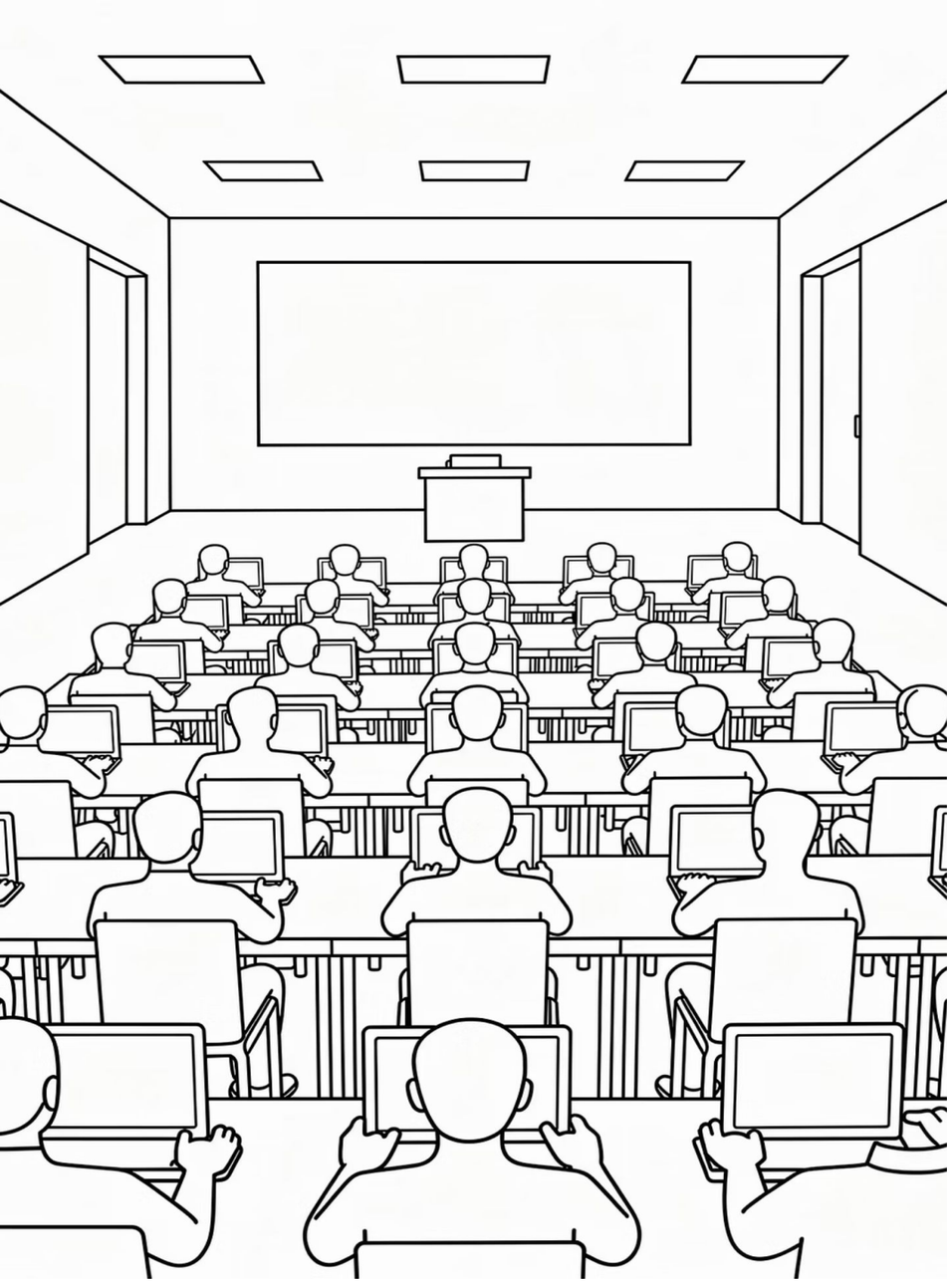


IIT JODHPUR

CPU Scheduling: Algorithms, Simulation & Analysis

Operating Systems · Week 3 · Live Practical Session · IIT Jodhpur

BS/BSc Programme · Semester 3

2-HOUR HANDS-ON SESSION

COPY.SH + GOOGLE COLAB

Futurense

IIT JODHPUR

Session Agenda

Here's how the next 120 minutes are structured — tools do the heavy lifting, slides just guide the way.

0

1

0–20 min — Setup & Rapid Quiz

Boot copy.sh, answer 8 memory-trigger questions on CPU burst, context switch, and preemption.

0

2

20–35 min — Analogy Walkthrough

Zomato, Railway, IPL, and Round Robin analogies mapped to OS concepts. Pair activity.

0

3

35–75 min — Activities 1 & 2 (copy.sh)

Observe real Linux processes, simulate FCFS and Round Robin via shell scripts.

0

4

75–120 min — Colab, Assignments & Wrap-up

Python Gantt charts, in-session assignments, exit ticket, and homework briefing.

Futurense

IIT JODHPUR

Prerequisites Check

Before we begin — confirm you have:

✅ Watched Week 3 Lecture (Part 1 & Part 2 fully)

✅ Browser tab open: **copy.sh/v86/?profile=buildroot**

✅ Google account ready for Colab

✅ Basic terminal commands from Weeks 1 & 2 (`ls`, `pwd`, `echo`)

TA Setup (Do Before Students Arrive)

- Open copy.sh and wait 30–60 sec for Linux to boot

Test with: `echo 'Terminal Ready'`

- Open Colab in a new tab — paste skeleton code into cells
- Display copy.sh URL on projector before students enter

Futurense

IIT JODHPUR

Rapid Revision Through Analogies

Before touching the terminal, let's reconnect concepts to the real world. Match each analogy to the OS term it represents.

🍕 Zomato Rider

CPU Burst vs IO Burst. Riding = CPU active. Waiting for food pack = IO Burst. Rider is **idle** during IO.

🚂 Railway Counter

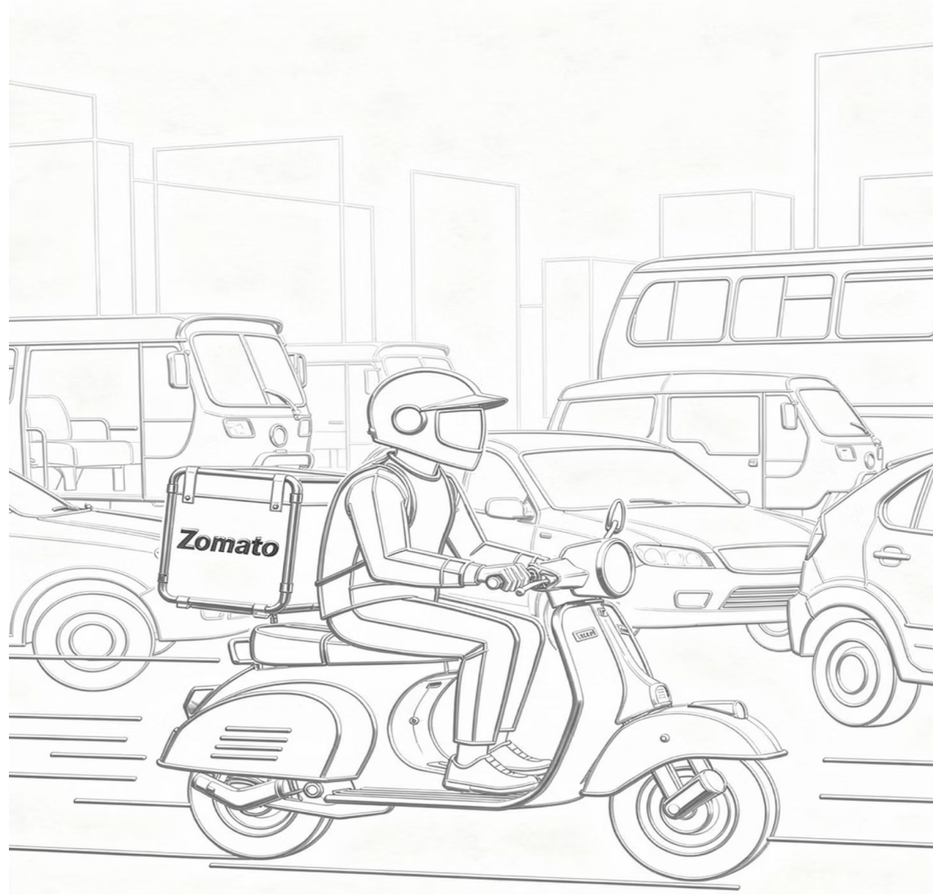
FCFS Scheduling. First passenger served first — even if their journey is 10 hours and the next person is going 2 stops.

🏏 IPL VIP Stands

Priority + Starvation. VIPs always eat first. Regular fans may wait the *entire match* without food.

🔄 Teacher's Doubt Round

Round Robin. 5 minutes per student, rotating until everyone is satisfied — equal quantum, guaranteed turn.



Futurense

Rapid Revision Quiz — 8 Questions

IIT JODHPUR

⏱️ Minutes 5–20

SHOW OF HANDS · VERBAL · NO MARKS

This is a **memory trigger**, not a test. Ask each question verbally, give 10–15 seconds, then reveal the answer together.

1

CPU Burst & IO Burst

Every process is interleaved with two types — name them. *(Answer: CPU burst + IO burst)*

2

Preemption

The professor used the word "snatching." What was snatched, from whom, and why?

3

FCFS Wait Time

P1=24ms, P2=3ms, P3=3ms, all arrive at t=0. Average waiting time? *(Answer: 17ms)*

4

Round Robin Formula

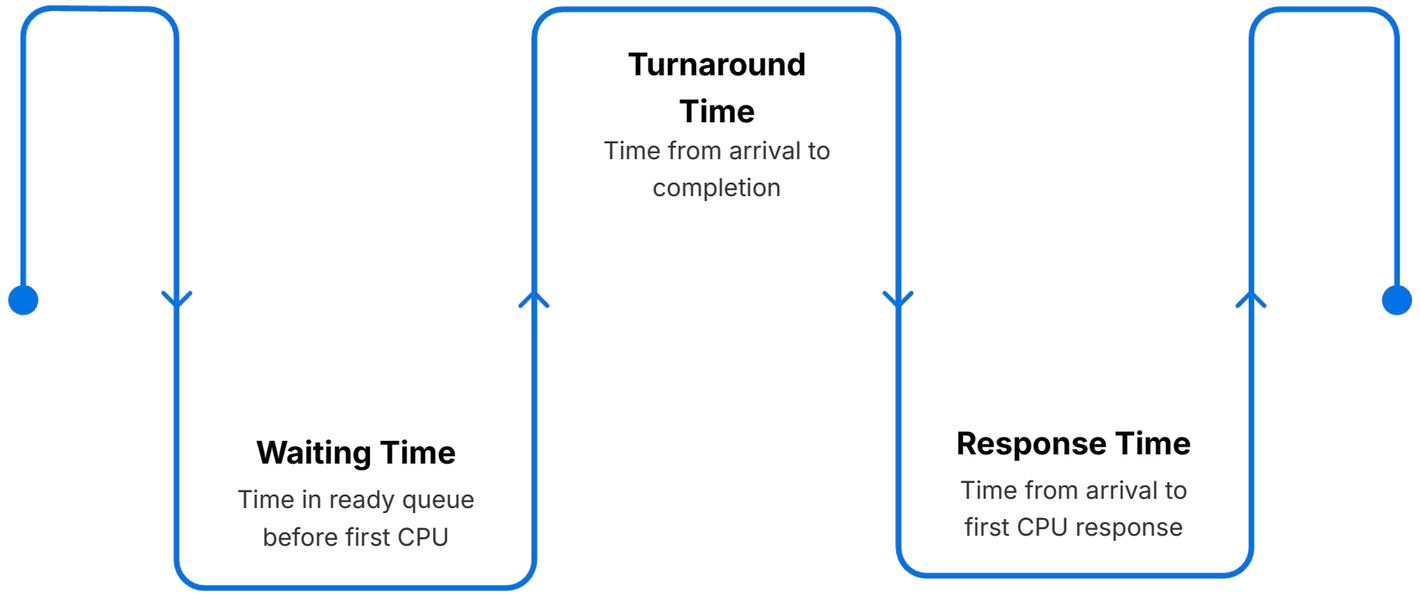
With quantum Q and N processes, max wait before getting CPU again? *(Answer: (N−1) × Q)*

Q5–Q8 cover SJF shortest burst, starvation in Priority Scheduling, and MLFQ demotion. Ask them verbally after these 4 warm-up questions.

Futurense

Core Scheduling Metrics — At a Glance

IIT JODHPUR



All three metrics use the same anchor: **Arrival Time**. The key formulas are WT = Start − Arrival, TAT = Finish − Arrival.

Common Pitfall ⚠

Students consistently confuse **Waiting Time** and **Turnaround Time**.

Waiting = time *sitting in queue*, not executing

Turnaround = total time from arrival *to completion* (includes execution)

Analogy: Waiting at the railway counter vs. total time from joining the queue to boarding the train.

Futurense

🖥️ ACTIVITY SLIDE

MINUTES 35–55 · COPY.SH TERMINAL

IIT JODHPUR

Activity 1 — Observing Real Linux Processes

Open **copy.sh/v86/?profile=buildroot** · Wait for the `$` or `#` prompt · Do NOT close this tab.

`echo 'Hello OS World'`

Confirms the terminal is alive. If it prints back — you're ready.

`ps aux`

Lists ALL running processes. Key columns: PID, STAT (R=Running, S=Sleeping, Z=Zombie), COMMAND.

`ps aux | wc -l`

Pipeline: list all processes → count lines. Output is the number of active processes.

`sleep 30 & sleep 20 & sleep 10 &`

Launches 3 background processes — simulating a **Ready Queue**. Then verify with `ps aux | grep sleep`.



Futurense

OPERATING SYSTEMS | WEEK 3

ACTIVITY 1 EXTENDED EXPLORATION

Getting Process State in BusyBox — Using /proc

BusyBox `ps aux` is limited, omitting critical process state information. To understand a process's true status (Running, Sleeping, Zombie), we must bypass userspace tools and read directly from the kernel via the **/proc** filesystem.

IIT JODHPUR

📁 What is /proc?

/proc is a **virtual filesystem** maintained by the Linux kernel entirely in memory. It provides real-time access to the kernel's internal data structures, giving each running process its own folder (`/proc/[PID]/`) containing live status files, process name, memory usage, and more. It's always available, making it the universal source for process information.

Process State Codes — Explained

The single-letter codes found in `/proc/[PID]/status` map directly to the process state diagram:

Code	State Name	What It Means
R	Running	Actively executing on the CPU (the RUNNING state).
S	Sleeping (interruptible)	Waiting for an event (IO, timer), can be woken up (the WAITING state).
D	Sleeping (uninterruptible)	Waiting for IO, CANNOT be interrupted or killed (e.g., disk wait).
Z	Zombie	Process finished, but parent hasn't collected exit status. Still in process table.
T	Stopped	Process paused by Ctrl+Z or a signal (e.g., SIGSTOP).

Key Methods for Observing Process States

Goal	Common Command
State of ONE process	`cat /proc/[PID]/status | grep State`
State of ALL processes	`for pid in /proc/[0-9]*; do cat $pid/status | grep State; done`
Compact state character	`cat /proc/[PID]/stat | awk '{print $3}'`
Scan for zombie processes	`for pid in /proc/[0-9]*; do cat $pid/status | grep -i zombie; done`
Create a zombie to observe	`sh -c '(exit 0) &; sleep 30'`

Futurense

Understanding the Pipeline

IIT JODHPUR



The Three-Stage Pipeline

```
ps aux | grep ash | wc -l
```

ps aux — produces all process rows

| grep ash — keeps only lines matching 'ash'

| wc -l — counts remaining lines

The `|` pipe is the glue. Output of one command becomes the *input* of the next — a fundamental Unix philosophy.

Pro tip: Use `kill [PID]` to terminate any sleep process. Then verify it's gone with `ps aux | grep sleep`.

Futurense

IIT JODHPUR

🖥️ ACTIVITY SLIDE | MINUTES 55–75 · SHELL SCRIPTS

Activity 2 — Scheduling Simulation via Shell Script

Students write and run shell scripts directly in copy.sh. Two scripts: **FCFS** and **Round Robin**.

How to Create a Script (Heredoc Syntax)

```
cat > fcfs.sh << 'EOF'
#!/bin/sh
# Your script here
EOF
chmod +x fcfs.sh
sh fcfs.sh
```

`cat >` redirects output to a file

`<< 'EOF'` opens a here-document — everything until `EOF` is the file content

`chmod +x` grants execute permission

`$(( expr ))` performs shell integer arithmetic — **always double brackets**

FCFS Script — Key Logic

```
CURRENT_TIME=0
START_P1=$CURRENT_TIME
CURRENT_TIME=$((CURRENT_TIME + BURST_P1))
FINISH_P1=$CURRENT_TIME
WT_P1=$((START_P1 - 0))
TAT_P1=$((FINISH_P1 - 0))
AVG_WT=$(((WT_P1+WT_P2+WT_P3)/3))
```

Result: P1 [0→24] | P2 [24→27] | P3 [27→30]

Average Waiting Time = **17ms**

Futurense

Activity 2 — Scheduling Simulation via Shell Script(FCFS)

IIT JODHPUR

```
cat > fcfs.sh << 'EOF'
#!/bin/sh
# ============================================================
# FCFS (First Come First Serve) Scheduling Simulator
# Based on the example from the lecture:
#   P1 burst=24, P2 burst=3, P3 burst=3, all arrive at t=0
# ============================================================

# Define process names as a space-separated string
PROCESSES="P1 P2 P3"

# Define burst times for each process (in milliseconds)
BURST_P1=24
BURST_P2=3
BURST_P3=3

# Initialize the current time (clock starts at 0)
CURRENT_TIME=0

echo "============================="
echo "  FCFS SCHEDULING SIMULATION"
echo "============================="
echo ""
echo "Gantt Chart:"
echo "---------------------------------"

# Process P1 first (it arrived first)
# Record when P1 STARTS executing
START_P1=$CURRENT_TIME

# P1 runs for its entire burst time without interruption (non-preemptive)
# CURRENT_TIME advances by P1's burst
CURRENT_TIME=$(( CURRENT_TIME + BURST_P1 ))

# Record when P1 FINISHES
FINISH_P1=$CURRENT_TIME

echo "| P1 [0 -> 24]             |"

# Process P2 second
START_P2=$CURRENT_TIME
CURRENT_TIME=$(( CURRENT_TIME + BURST_P2 ))
FINISH_P2=$CURRENT_TIME
echo "| P2 [24 -> 27]            |"

# Process P3 last
START_P3=$CURRENT_TIME
CURRENT_TIME=$(( CURRENT_TIME + BURST_P3 ))
FINISH_P3=$CURRENT_TIME
echo "| P3 [27 -> 30]            |"
echo "---------------------------------"
echo ""

# Calculate Waiting Time for each process
# Formula: Waiting Time = Start Time - Arrival Time
# Since all arrive at t=0: Waiting Time = Start Time
WT_P1=$(( START_P1 - 0 ))
WT_P2=$(( START_P2 - 0 ))
WT_P3=$(( START_P3 - 0 ))

# Calculate Turnaround Time for each process
# Formula: Turnaround Time = Finish Time - Arrival Time
TAT_P1=$(( FINISH_P1 - 0 ))
TAT_P2=$(( FINISH_P2 - 0 ))
TAT_P3=$(( FINISH_P3 - 0 ))

echo "Results:"
echo "Process | Burst | Start | Finish | Wait | Turnaround"
echo "--------|-------|-------|--------|------|-----------"
echo "P1      |  $BURST_P1   |   $START_P1   |   $FINISH_P1   |  $WT_P1   |  $TAT_P1"
echo "P2      |   $BURST_P2   |  $START_P2   |   $FINISH_P2   |  $WT_P2  |  $TAT_P2"
echo "P3      |   $BURST_P3   |  $START_P3   |   $FINISH_P3   |  $WT_P3  |  $TAT_P3"
echo ""

# Calculate Average Waiting Time
# $(( )) is shell arithmetic — evaluates integer math expressions
AVG_WT=$(( (WT_P1 + WT_P2 + WT_P3) / 3 ))
echo "Average Waiting Time = ($WT_P1 + $WT_P2 + $WT_P3) / 3 = $AVG_WT ms"
EOF
chmod +x fcfs.sh
sh fcfs.sh
```

Futurense

Activity — Scheduling- Script(Round Robin)

IIT JODHPUR

```
cat > rr.sh << 'EOF'
#!/bin/sh
# ============================================================
# Round Robin Scheduling Simulator
# Processes: P1=24ms, P2=3ms, P3=3ms  Time Quantum=4ms
# Demonstrates how RR gives equal CPU time slices to all
# ============================================================

# Time quantum = the fixed slice of CPU time given to each
process
QUANTUM=4

# Remaining burst times for each process
# These will DECREASE each time a process gets CPU time
REM_P1=24
REM_P2=3
REM_P3=3

# Starting clock
TIME=0

echo "================================"
echo "  ROUND ROBIN SCHEDULING (Q=4)"
echo "================================"
echo ""
echo "Time Quantum = $QUANTUM ms"
echo ""
echo "Gantt Chart:"
echo "----------------------------------------"

# Round 1: Each process gets one time quantum
# P1 has 24ms remaining, gets 4ms
echo "| P1 [0 -> 4] (remaining: 20ms)"
TIME=$(( TIME + QUANTUM ))
REM_P1=$(( REM_P1 - QUANTUM ))

# P2 has only 3ms remaining — less than quantum!
# P2 finishes in this turn (3ms, not 4ms)
P2_SLICE=3
echo "| P2 [4 -> 7] (remaining: 0ms — DONE!)"
TIME=$(( TIME + P2_SLICE ))
REM_P2=0

# P3 also only needs 3ms
P3_SLICE=3
echo "| P3 [7 -> 10] (remaining: 0ms — DONE!)"
TIME=$(( TIME + P3_SLICE ))
REM_P3=0

# P2 and P3 are done. Only P1 remains with 20ms
# P1 gets the rest of the CPU time
echo "| P1 [10 -> 30] (P1 finishes — DONE!)"
TIME=$(( TIME + REM_P1 ))
REM_P1=0

echo "----------------------------------------"
echo ""

# Now calculate metrics
# Arrival time = 0 for all, so Waiting Time = Start Time (approximately)
echo "Completion Times:"
echo "P2 finished at: 7ms"
echo "P3 finished at: 10ms"
echo "P1 finished at: 30ms"
echo ""

# Turnaround Time = Completion Time - Arrival Time (arrival=0)
echo "Turnaround Times:"
echo "P1 TAT = 30 - 0 = 30ms"
echo "P2 TAT = 7 - 0 = 7ms"
echo "P3 TAT = 10 - 0 = 10ms"
echo ""
echo "Average TAT = (30 + 7 + 10) / 3 = $(( (30+7+10)/3 )) ms"
echo ""

# KEY INSIGHT about Round Robin:
echo "KEY INSIGHT:"
echo "With Q=$QUANTUM and 3 processes:"
echo "Max wait before getting CPU again = (N-1) x Q = 2 x 4 = 8ms"
echo "This GUARANTEES response time — unlike FCFS where P3 waits 27ms!"
EOF
chmod +x rr.sh
sh rr.sh
```

Futurense

IIT JODHPUR

FCFS vs Round Robin — Side by Side

FCFS (Non-Preemptive)

```
| P1 [0  -> 24] |
| P2 [24 -> 27] |
| P3 [27 -> 30] |

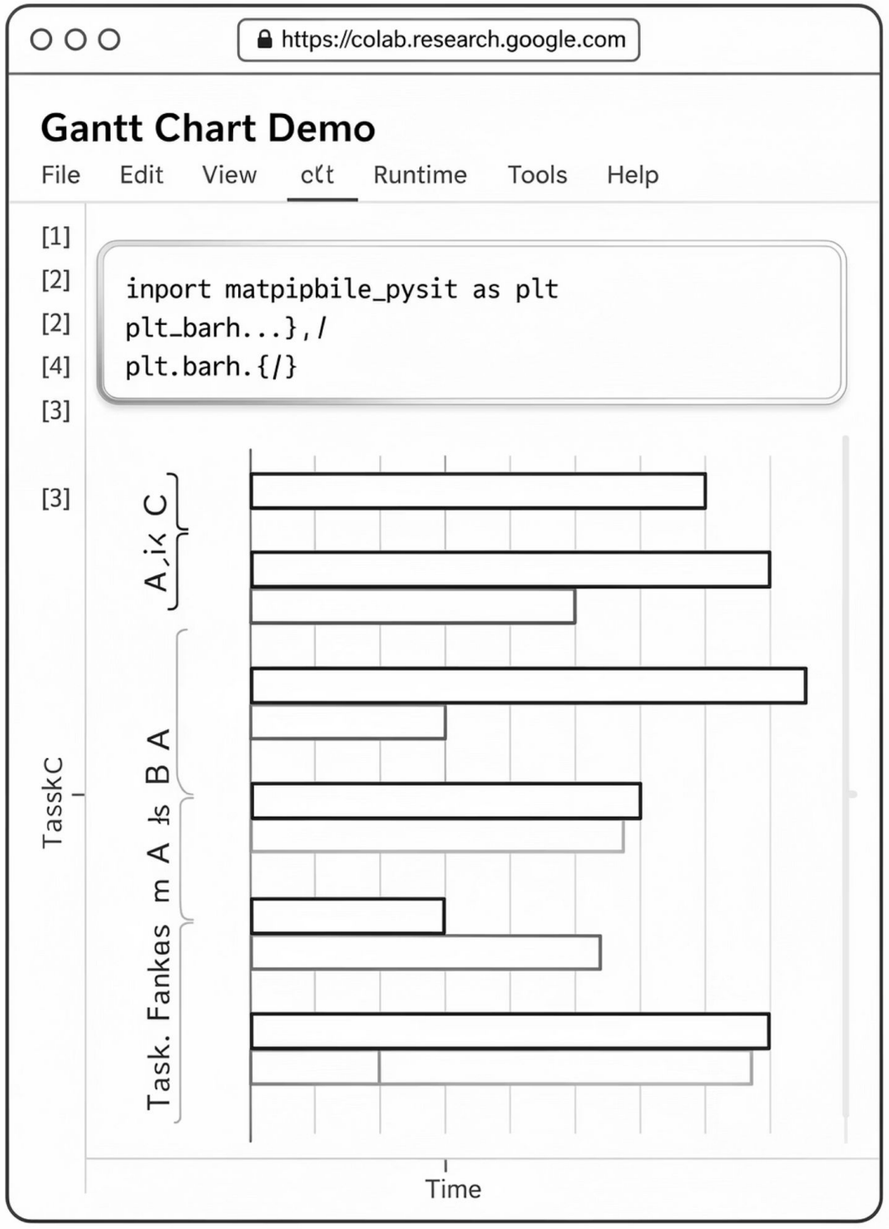

Avg Waiting Time = 17ms
P2 waits 24ms — painful!
```

Fair by arrival order. But short jobs suffer behind long ones — the **convoy effect**.

Round Robin (Q = 4ms)

```
| P1 [0 -> 4] (rem: 20ms)
| P2 [4 -> 7] (DONE)
| P3 [7 -> 10] (DONE)
| P1 [10 -> 30] (DONE)


Max wait = (N-1)×Q = 8ms
```



Futurense

🐍 ACTIVITY SLIDE

MINUTES 75–90 · GOOGLE COLAB

IIT JODHPUR

Activity 3 — Scheduling Visualiser in Google Colab

Open **colab.research.google.com** · New Notebook · Press **Shift+Enter** to run each cell.

0

1

Cell 1 — Imports & Process Table

Import `matplotlib`, `numpy`. Define 4 processes as Python dicts with `name`, `arrival`, `burst`, `priority`.

0

2

Cell 2 — simulate_fcfs()

Sort by arrival time. Non-preemptive: each process runs its full burst. Returns Gantt blocks + waiting/turnaround metrics.

0

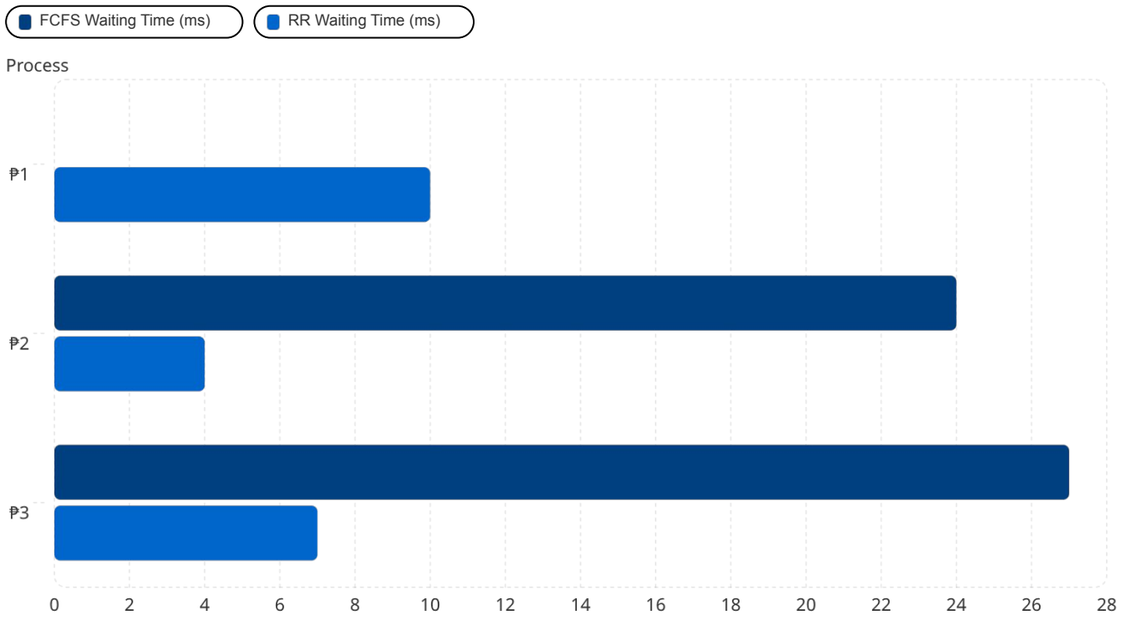
3

Cell 3 — simulate_round_robin() + draw_gantt()

Circular queue with `quantum=4`. Uses `ax.barh()` to draw colour-coded Gantt bars. Runs both algorithms and overlays them for comparison.

IIT JODHPUR

Reading the Gantt Chart Output



What This Chart Tells You

FCFS is catastrophic for P2 and P3 — they wait **24ms and 27ms** respectively while P1 monopolizes the CPU.

Round Robin caps maximum wait at **(N−1) × Q = 8ms**, guaranteeing responsiveness for all processes.

- Lower waiting time → better user experience
- Trade-off: RR adds more context switches
- Colab lets you tune Q and instantly see the impact

Futurense

✍ IN-SESSION ASSIGNMENT

MINUTES 90–100 · INDIVIDUAL WORK

IIT JODHPUR

Assignments — Work Independently

Assignment 2 — Priority Scheduling + Starvation (priority.sh)

Processes: P1(pri=3), P2(pri=1), P3(pri=4), P4(pri=5), P5(pri=1).

Execute lowest-number-first. Show P4 waiting the longest. Add a comment explaining **why P4 starves** and how **aging** solves it.

- Define the

  Priority order, provide Starvation explanation

-

Futurense

Common Pitfalls & Debug Session

IIT JODHPUR

MINUTES 100–110

⚠ Script not found / EOF error

The last line of the heredoc must be *exactly* `EOF` — no trailing spaces. Retype from scratch if needed.

⚠ Wrong arithmetic results

Shell arithmetic **requires double brackets**: `$(( expr ))`. Single bracket `$( )` is command substitution — it does NOT do math.

⚠ Colab chart not showing

Make sure `plt.show()` is called at the end of the cell. Add `%matplotlib inline` at the very top of the notebook.

⚠ SJF "knows the future" misconception

SJF is **optimal in theory** but impractical — we can't know burst times in advance. The OS *predicts* using exponential averaging: *\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n*

Futurense

Exit Ticket — 3 Questions in 5 Minutes

IIT JODHPUR

📝 Assessment

MINUTES 110–115

Answer in a Colab text cell or on paper. TA will collect before dismissal.

1

Concept Check

P1 arrival=0 burst=10, P2 arrival=0 burst=4, P3 arrival=0 burst=6. Compute FCFS avg waiting time AND SJF avg waiting time. Which is better — and why?

2

Analogy Application

You're building a food delivery app with 3 delivery agents. Orders vary (1 item to 10 items). Which scheduling algorithm do you use — FCFS, SJF, RR, or Priority? Justify your choice.

3

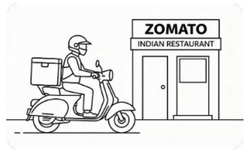
copy.sh Practical

Write ONE terminal command that lists all processes, filters for user 'root', and counts how many there are. *(Hint: combine* `ps aux`*,* `grep`*, and* `wc`*.)*

Futurense

IIT JODHPUR

TA Quick Reference — Analogy Cheat Sheet



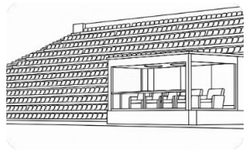
🍕 Zomato — CPU vs IO Burst

Riding = CPU burst. Waiting for food pack = IO burst. The CPU (rider) should **never sit idle** — assign another process!



🚂 Railway — FCFS

First in line, first served. Convoy effect: a 10-hour journey blocks everyone behind — even a 2-stop traveller.



🏏 IPL VIP — Priority + Starvation

VIPs always eat first. General fans may go hungry the whole match. **Aging** = after 30 minutes, fan gets promoted to VIP.



🔄 Teacher's Round — Round Robin

5 minutes per student, round and round. Equal quantum. Guarantees every student gets a turn — fair and predictable.

Futurense

📚 Homework — Before Next Session

IIT JODHPUR

1 Part A — RR with Q=2ms

Modify `rr.sh` to use quantum=2ms. Compare output: does avg waiting time change? Count context switches.

2 Part B — Priority in Colab

Implement `simulate_priority()` in Colab (sort by priority ascending). Generate Gantt chart. Compare avg WT for FCFS, SJF, RR, and Priority on the same 4 processes.

3 Part C — Research (Straggler Problem)

The professor linked CPU scheduling to DNN training. In 2–3 sentences: how can scheduling decisions slow down a neural network training job?

TA Timing Checklist ✅

Activity	Status
Setup + Quiz (0–20 min)	✅ All 8 Qs done
Analogies (20–35 min)	✅ 4 covered
Activity 1 (35–55 min)	✅ 8 commands run
Activity 2 (55–75 min)	✅ fcfs.sh + rr.sh
Activity 3 (75–90 min)	✅ Gantt visible
Assignments (90–100 min)	✅ A1 skeleton filled
Exit Ticket (110–115 min)	✅ Collected

Session Complete — Key Takeaways

IIT JODHPUR

Algorithms are Trade-offs

No single scheduler is universally best. FCFS is simple but unfair. SJF is optimal but impractical. RR is fair but context-switch heavy.

Metrics Matter

Always compute WT, TAT, and Response Time. The same process set can yield radically different metrics under different schedulers.

Simulation = Understanding

Running a shell script or Colab cell makes the abstract concrete. You can't truly understand MLFQ until you watch a process get demoted.

Starvation is Real

Priority scheduling without aging is broken by design. The IPL analogy is not just fun — it's the actual problem. Aging is the real fix.

Next session: **Memory Management** — paging, segmentation, and virtual memory. Same tools, new concepts. See you there! 🚀

Futurense